

Developing with the Cerebro SDK

A Practical Guide for Engineers

Writer Security Engineering

Updated November 6, 2025

What You'll Learn

- ▶ Why the Cerebro SDK exists and what problems it solves
- ▶ Getting started: installation, configuration, authentication
- ▶ Building with core modules: agents, findings, analytics, security center
- ▶ Advanced patterns: streaming, pagination, testing strategies
- ▶ Best practices for production deployments

Why Use the Cerebro SDK?

The Problem:

- ▶ Ad-hoc REST calls scattered across services
- ▶ No type safety, manual JSON parsing
- ▶ Duplicate pagination and error handling logic
- ▶ Difficult to test without hitting live APIs

The SDK Solution:

- ▶ Typed interfaces for all Cerebro operations
- ▶ Consistent patterns for async workflows
- ▶ Built-in observability and error handling
- ▶ Test adapters for offline development

Two SDKs, One Platform

TypeScript SDK (`sdk/ts`)

- ▶ Frontend integrations
- ▶ Browser-based tools
- ▶ Partner APIs
- ▶ Real-time streaming

Python SDK (`src/cerebro_sdk`)

- ▶ Backend automation
- ▶ Compliance tooling
- ▶ Async workflows
- ▶ Database-backed operations

Both SDKs share:

- ▶ Identical domain models (agents, findings, evidence)
- ▶ Same API surface and semantics
- ▶ Parallel feature development

Core Capabilities

What the SDK Provides:

- ▶ **Agent Operations:** sessions, memory, review workflows, tool execution
- ▶ **Security Intelligence:** findings, analytics, runtime health
- ▶ **Compliance:** control testing, evidence management, audit workflows
- ▶ **Integration Management:** provider sync, coverage tracking
- ▶ **Observability:** structured logging, Prometheus metrics

Python SDK: Installation

From monorepo

```
uv pip install -e /path/to/cerebro
```

Or via path dependency in pyproject.toml

```
[tool.poetry.dependencies]
```

```
cerebro-sdk = { path = "../cerebro", develop = true }
```

Verify installation

```
python -c "from cerebro_sdk import AuthSession; print('OK')"
```

Requirements:

- ▶ Python 3.11+
- ▶ SQLAlchemy async session (provided by Cerebro core)
- ▶ PostgreSQL database connection

TypeScript SDK: Installation

```
# Install from internal registry  
npm install @writerai/cerebro-sdk
```

```
# Or local development  
cd sdk/ts  
npm install  
npm run build
```

```
# Link for local testing  
npm link  
cd your-project  
npm link @writerai/cerebro-sdk
```

Requirements:

- ▶ Node.js 18+ (native fetch support)
- ▶ TypeScript 5.0+ for full type inference
- ▶ Modern bundler (Vite, Webpack, esbuild)

Python: Basic Setup

```
from cerebro.core.database import async_session_factory
from cerebro_sdk import AuthSession, AgentManager

async def example():
    async with async_session_factory() as db:
        # Authenticate
        auth = AuthSession(db)
        tokens = await auth.login("username", "password")

        # Use SDK
        agents = AgentManager(db)
        sessions = await agents.list_sessions(
            org_id="org-123",
            limit=10
        )

        for session in sessions:
            print(session.session_id, session.agent_type)
```


TypeScript: Basic Setup

```
import { CerebroSDK } from "@writerai/cerebro-sdk";

const sdk = new CerebroSDK({
  baseUrl: process.env.CEREBRO_BASE_URL!,
  async getAccessToken() {
    return await authClient.getToken();
  },
  defaultHeaders: {
    "x-writer-service": "my-automation"
  }
});

// Use the SDK
const { sessions } = await sdk.agents.listSessions({
  agentType: "security_investigator",
  limit: 10
});

sessions.forEach(s => console.log(s.sessionId));
```

Authentication

Python: Database-backed Auth

```
from cerebro_sdk import AuthSession

auth = AuthSession(db)
tokens = await auth.login("analyst", "password")
print(tokens.access_token, tokens.refresh_token)

# Verify token
payload = await auth.verify(
    tokens.access_token,
    expected_type="access"
)
print(payload["username"], payload["scopes"])
```

TypeScript: Token Provider Pattern

```
const sdk = new CerebroSDK({
  baseUrl: "https://cerebro.writer.com",
  async getAccessToken() {
    // Fetch from your auth service
```

Working with Agents: Creating Sessions

Python:

```
from cerebro_sdk import AgentManager

agents = AgentManager(db)
session = await agents.create_session(
    org_id="org-123",
    agent_type="security_analyst",
    created_by="automation@writer.com",
    context={"finding_ids": ["finding-abc"]}
)

await agents.add_message(
    session_id=session.session_id,
    role="user",
    content={"text": "Summarize this finding"}
)
```

TypeScript:

```
const session = await sdk.agents.createSession({
```

Working with Agents: Streaming Responses

TypeScript Streaming:

```
import { collectAgentStream } from "@writerai/cerebro-sdk";

const stream = sdk.agents.streamSession({
  sessionId: session.sessionId,
  message: "Analyze the blast radius"
});

const result = await collectAgentStream(stream);
console.log("Completion:", result.completion);
console.log("Tool calls:", result.toolCalls.length);
console.log("Errors:", result.errors);
```

The SDK handles:

- ▶ SSE parsing and reconnection
- ▶ Type guards for different event types
- ▶ Automatic accumulation of partial responses

Review Queue Management

Python: Bulk Operations

```
from cerebro_sdk import AgentReviewManager
```

```
review = AgentReviewManager(db)
```

```
# Get pending tasks
```

```
tasks = await review.list_tasks(  
    org_id="org-123",  
    status="pending",  
    limit=50  
)
```

```
# Bulk update
```

```
await review.bulk_update_tasks(  
    task_ids=[t.task_id for t in tasks[:10]],  
    status="in_progress",  
    assigned_to="analyst@writer.com"  
)
```

Findings and Analytics

Python:

```
from cerebro_sdk import FindingService, RuntimeHealthClient

# List findings
findings_svc = FindingService(db)
findings = await findings_svc.list_findings(
    org_id="org-123",
    severity="critical",
    status="open"
)

# Runtime health metrics
health = RuntimeHealthClient(db)
metrics = await health.get_runtime_health(
    org_id="org-123",
    since_hours=24
)

print(f"Events: {metrics.total_events}")
```

Security Center: Vendor Management

TypeScript:

```
// List vendors with risk filtering
```

```
const vendors = await sdk.securityCenter.listVendors(  
  "org-123",  
  { riskLevel: "high", category: "security" }  
);
```

```
// Register new vendor
```

```
await sdk.securityCenter.registerVendor("org-123", {  
  name: "Acme Cloud",  
  websiteUrl: "https://acme.example.com",  
  category: "security",  
  businessCriticality: "high"  
});
```

```
// Assess vendor health
```

```
import { assessVendorHealth } from "@writerai/cerebro-sdk";
```

```
const health = vendors.map(v => assessVendorHealth(v));
```

Pagination: Python

```
from cerebro_sdk import AgentManager

agents = AgentManager(db)

# Manual pagination
cursor = None
all_sessions = []

while True:
    result = await agents.list_sessions(
        org_id="org-123",
        cursor=cursor,
        limit=100
    )
    all_sessions.extend(result.sessions)

    if not result.next_cursor:
        break
    cursor = result.next_cursor
```


Pagination: TypeScript

```
import { iterateCursor, collectCursor }  
  from "@writerai/cerebro-sdk";  
  
// Stream with async iteration  
for await (const session of iterateCursor(  
  () => sdk.agents.listSessions({ orgId: "org-123" })  
)) {  
  await processSession(session);  
}  
  
// Or collect all at once  
const allSessions = await collectCursor(  
  () => sdk.agents.listSessions({ orgId: "org-123" })  
);  
  
console.log(`Total: ${allSessions.length}`);
```

Tool Execution and Approvals

Python:

```
from cerebro_sdk import AgentToolingManager
from cerebro.agents.models import ToolInvocationStatus

tooling = AgentToolingManager(db)

# Create tool invocation
invocation = await tooling.create_invocation(
    session_id=session.session_id,
    tool_name="remediation.isolate_host",
    input_data={"host_id": "host-123"},
    status=ToolInvocationStatus.PENDING_APPROVAL
)

# Approve and execute
updated = await tooling.update_invocation_result(
    invocation_id=invocation.invocation_id,
    status=ToolInvocationStatus.SUCCESS,
    output_data={"isolated": True},
```

Evidence Lifecycle Management

Both SDKs Support Evidence Primitives:

```
from cerebro_sdk.security_center import (
    extract_evidence_artifacts,
    evaluate_evidence_lifecycle,
    LifecyclePolicy
)

# Extract evidence from metadata
metadata = {
    "evidence": {
        "id": "soc2-audit",
        "collected_at": "2024-08-01T00:00:00Z"
    }
}

artifacts = extract_evidence_artifacts(
    kind="vendor",
    entity_id="vendor-1",
    metadata=metadata
```

Observability: Logging

Python: Structured Logging

```
from cerebro_sdk import configure_logging

logger = configure_logging(
    level="INFO",
    json_format=True,
    include_timestamp=True
)

logger.info(
    "Agent session created",
    extra={
        "session_id": session.session_id,
        "org_id": org_id,
        "agent_type": "security_analyst"
    }
)
```

TypeScript: Custom Middleware

Observability: Metrics

Python: Prometheus Integration

```
from prometheus_client import CollectorRegistry
from cerebro_sdk import AgentToolingManager

registry = CollectorRegistry()
tooling = AgentToolingManager(db, registry=registry)

# Operations automatically emit metrics
await tooling.list_invocations(org_id="org-123")

# Metrics available:
# - cerebro_sdk_tool_invocation_queries_total
# - cerebro_sdk_tool_approval_queries_total
# - cerebro_sdk_notifications_total
# - cerebro_sdk_tickets_total
```

Use these metrics to:

- ▶ Track API call volume and latency

Python: In-Memory Testing

```
import pytest
from cerebro_sdk.testing import InMemoryAgentManager

@pytest.fixture
def agent_manager():
    return InMemoryAgentManager()

async def test_session_creation(agent_manager):
    session = await agent_manager.create_session(
        org_id="test-org",
        agent_type="analyst",
        created_by="test@example.com"
    )

    assert session.session_id is not None
    assert session.agent_type == "analyst"

    # Operations work without database
    sessions = await agent_manager.list_sessions()
```

TypeScript: Contract Testing

```
import { describe, it, expect } from "vitest";
import { CerebroSDK } from "@writerai/cerebro-sdk";

describe("Agents Client", () => {
  const sdk = new CerebroSDK({
    baseUrl: "http://localhost:8000",
    getAccessToken: async () => "test-token"
  });

  it("creates and lists sessions", async () => {
    const session = await sdk.agents.createSession({
      orgId: "test-org",
      agentType: "analyst"
    });

    expect(session.sessionId).toBeDefined();

    const { sessions } = await sdk.agents.listSessions({
      orgId: "test-org"
```

Testing Best Practices

Unit Tests:

- ▶ Use in-memory adapters (Python) or mocked HTTP clients (TypeScript)
- ▶ Test business logic without external dependencies
- ▶ Fast feedback loop (< 1 second per test)

Integration Tests:

- ▶ Point at local Cerebro instance or staging environment
- ▶ Test full workflows end-to-end
- ▶ Verify SDK behavior matches API contracts

Contract Tests:

- ▶ Ensure TypeScript and Python SDKs produce equivalent results
- ▶ Validate serialization and deserialization
- ▶ Catch API schema drift early

Error Handling Strategy

SDK Exceptions Hierarchy:

- ▶ `AgentSDKError` (base for all SDK errors)
 - ▶ `AgentNotFoundError`: resource doesn't exist
 - ▶ `AgentInvalidStatusError`: invalid state transition
 - ▶ `AgentValidationError`: input validation failed
- ▶ `HttpError` (`TypeScript`): HTTP request failures

Recommended Pattern:

- ▶ Catch specific exceptions for known failure modes
- ▶ Log `AgentSDKError` base class for unexpected errors
- ▶ Include context (`org_id`, `session_id`) in error logs
- ▶ Retry on transient network errors, not validation errors

Retry Pattern

TypeScript with Exponential Backoff:

```
import { CerebroSDK, HttpError } from "@writera/cerebro-sdk";

async function withRetry<T>(<
  fn: () => Promise<T>,
  maxAttempts = 3
>): Promise<T> {
  for (let attempt = 1; attempt <= maxAttempts; attempt++) {
    try {
      return await fn();
    } catch (error) {
      if (error instanceof HttpError &&
        error.status >= 500 &&
        attempt < maxAttempts) {
        const delay = Math.pow(2, attempt) * 1000;
        await new Promise(r => setTimeout(r, delay));
        continue;
      }
      throw error;
    }
  }
}
```

Connection Pool Management

Python: Database Session Best Practices

```
from cerebro.core.database import async_session_factory
from contextlib import asynccontextmanager
```

```
@asynccontextmanager
async def get_sdk_session():
    """Reusable session context manager"""
    async with async_session_factory() as db:
        yield db
```

```
# In your service
async def process_findings(org_id: str):
    async with get_sdk_session() as db:
        findings = FindingService(db)
        results = await findings.list_findings(
            org_id=org_id,
            severity="critical"
        )
    return results
```

Rate Limiting and Throttling

SDK Built-in Protections:

- ▶ Respects HTTP 429 responses with Retry-After headers
- ▶ Configurable timeout per request (default: 30s)
- ▶ Connection pooling prevents overwhelming backend

Application-Level Strategies:

- ▶ Use pagination limits to control batch sizes
- ▶ Implement job queues for bulk operations
- ▶ Cache frequently accessed data (findings, org metadata)
- ▶ Monitor SDK metrics for throttling signals

Recommended Limits:

- ▶ Findings/sessions: 100 per page
- ▶ Review tasks: 50 per page
- ▶ Concurrent API calls: 10 per service

Configuration Management

Python: Settings Proxy Pattern

- ▶ Cache settings in-memory with automatic refresh
- ▶ Environment variable overrides
- ▶ Safe defaults for development vs. production

TypeScript: Environment-Aware Setup

- ▶ Different base URLs per environment
- ▶ Token providers vary by runtime (browser vs. Node.js)
- ▶ Custom headers for service identification

Secrets Management:

- ▶ Never hardcode API keys or passwords
- ▶ Use environment variables or secret managers
- ▶ Rotate tokens regularly, SDK supports async token refresh

Example: Automated Finding Triage

```
from cerebro_sdk import FindingService, AgentManager

async def auto_triage_critical_findings(org_id: str):
    async with get_sdk_session() as db:
        findings_svc = FindingService(db)
        agents = AgentManager(db)

        # Get new critical findings
        findings = await findings_svc.list_findings(
            org_id=org_id,
            severity="critical",
            status="open",
            limit=10
        )

        for finding in findings:
            # Create agent session for each
            session = await agents.create_session(
                org_id=org_id,
```

Example: Compliance Report Generation

```
import { CerebroSDK, assessVendorHealth }
  from "@writerai/cerebro-sdk";

async function generateComplianceReport(orgId: string) {
  const sdk = new CerebroSDK({...});

  // Fetch all vendors
  const vendors = await sdk.securityCenter.iterateVendors(
    orgId,
    { limit: 100 }
  );

  // Assess health
  const assessments = vendors.map(assessVendorHealth);

  const overdue = assessments.filter(
    a => a.reviewStatus === "overdue"
  );
  const expiringSoon = assessments.filter(
```

Example: Review Queue Dashboard

```
from cerebro_sdk import AgentReviewManager

async def get_review_queue_metrics(org_id: str):
    async with get_sdk_session() as db:
        review = AgentReviewManager(db)

        # Get all pending tasks
        pending = await review.list_tasks(
            org_id=org_id, status="pending"
        )

        # SLA at-risk tasks
        at_risk = await review.list_tasks_sla_at_risk(
            org_id=org_id, threshold_hours=24
        )

        # Summary stats
        summary = await review.get_queue_summary(org_id)
```


Common Issues: Python

Symptom	Solution
ModuleNotFoundError	Ensure SDK is installed: <code>uv pip install -e .</code>
AgentNotFoundError	Verify resource exists, check <code>org_id</code> scope
Session pool exhausted	Reduce concurrent operations, check for leaks
Slow queries	Add database indexes, use pagination
Type errors	Update to latest SDK, check model definitions

Common Issues: TypeScript

Symptom	Solution
<code>fetch</code> is not a function	Upgrade to Node 18+ or add <code>undici</code> polyfill
<code>HttpError</code> with empty body	Server returned non-JSON, check logs
SSE stream terminates early	Increase timeout, check network stability
Type mismatches	Regenerate adapters from OpenAPI spec
Module not found	Check bundler config, ensure ESM support

Debugging Techniques

Enable Debug Logging:

- ▶ Python: `configure_logging(level="DEBUG")`
- ▶ TypeScript: Add `beforeRequest/afterResponse` middleware

Inspect Raw API Responses:

- ▶ Python: Check `AsyncSession.execute()` SQL output
- ▶ TypeScript: Use browser DevTools Network tab

Test in Isolation:

- ▶ Use in-memory adapters to rule out API issues
- ▶ Try equivalent curl commands to verify API behavior
- ▶ Check Cerebro logs for server-side errors

Version Compatibility:

- ▶ SDK version must match API version
- ▶ Check `CHANGELOG.md` for breaking changes

Development Best Practices

Code Organization:

- ▶ Create service wrappers around SDK clients
- ▶ Don't pass raw database sessions to business logic
- ▶ Use dependency injection for testability

Error Handling:

- ▶ Always catch SDK-specific exceptions
- ▶ Log errors with sufficient context
- ▶ Don't retry validation errors, only transient failures

Performance:

- ▶ Use pagination for large result sets
- ▶ Cache frequently accessed data
- ▶ Monitor database connection pool usage

Production Best Practices

Observability:

- ▶ Emit metrics for SDK operations (counters, histograms)
- ▶ Log request IDs for distributed tracing
- ▶ Alert on error rate spikes

Security:

- ▶ Use short-lived access tokens
- ▶ Rotate credentials regularly
- ▶ Validate org_id scope before operations

Reliability:

- ▶ Implement retry logic with exponential backoff
- ▶ Set appropriate timeouts (30-60s for most operations)
- ▶ Have fallback behavior for degraded API states

Testing Best Practices

Unit Testing:

- ▶ Use in-memory adapters, not live APIs
- ▶ Test edge cases and error paths
- ▶ Keep tests fast ($< 100\text{ms}$ per test)

Integration Testing:

- ▶ Test against local Cerebro instance
- ▶ Clean up test data after runs
- ▶ Use dedicated test organizations

Contract Testing:

- ▶ Verify SDK matches API contracts
- ▶ Test serialization/deserialization
- ▶ Catch schema drift early

Documentation

In-Repo Guides:

- ▶ `docs/sdk/README.md` - SDK overview
- ▶ `docs/sdk/python.md` - Python-specific guide
- ▶ `docs/sdk/typescript.md` - TypeScript-specific guide
- ▶ `docs/sdk/auth-and-agents.md` - Agent workflows
- ▶ `docs/sdk/integrations-and-playbooks.md` - Advanced patterns

Examples:

- ▶ `examples/` - End-to-end scenarios
- ▶ `tests/unit/sdk/` - Python unit tests
- ▶ `sdk/ts/test/` - TypeScript test suite

Getting Help

Internal Channels:

- ▶ Slack: `#cerebro-sdk` for SDK-specific questions
- ▶ Slack: `#cerebro-general` for broader platform questions
- ▶ Office hours: Tuesdays 2-3pm with SDK maintainers

Reporting Issues:

- ▶ GitHub issues with `sdk` label
- ▶ Include SDK version, code sample, error message
- ▶ Provide minimal reproducible example

Contributing:

- ▶ PRs welcome for bug fixes and new features
- ▶ Follow existing patterns and test coverage
- ▶ Update documentation for new capabilities

Next Steps

Getting Started:

1. Install SDK in your project (Python or TypeScript)
2. Set up authentication and configuration
3. Build a simple integration (list sessions or findings)
4. Add error handling and logging

Go Deeper:

- ▶ Explore streaming APIs for real-time updates
- ▶ Implement pagination for large datasets
- ▶ Add observability with metrics and tracing
- ▶ Build production-ready retry and fallback logic

Share Your Work:

- ▶ Document your integration patterns
- ▶ Share reusable SDK wrappers with the team
- ▶ Present your use case at weekly brownbag

Questions?

Contact the SDK team:

Slack: #cerebro-sdk

Email: cerebro-sdk@writer.com

Office Hours: Tuesdays 2-3pm

This presentation: `docs/talks/cerebro-sdk-developer-guide.tex`